General Requirements for All Tasks

To ensure a robust and scalable solution, all projects must adhere to the following general requirements:

- **Implementation**
  - Implement pattern-specific classes (e.g., CacheMapper, Partitioner, or custom paired Mappers) following best practices.
- **Mapper Logic**
  - Extract the appropriate key and value from each input record, ensuring efficient data processing.
- **Reducer Logic**
  - Aggregate or enrich values for each key, producing the desired output.
- **Driver Class**
  - Configure and submit the MapReduce job using a Driver class, specifying correct reducers, input/output paths, and necessary registrations (e.g., DistributedCache, MultipleInputs, CustomPartitioner).
- **Large File Handling**
  - Solutions must be capable of processing datasets of 1 GB or larger.
- **Data Validation**
  - Robust error handling is mandatory for malformed lines, missing fields, and unparseable numeric values.
- **Testing**
  - Verify against provided sample data and real-world datasets.
- **Submission**
  - All Java source files (.java), a README file, sample input/output files, documented assumptions, and well-commented code.
- **Combiner Class Usage (Conditional)**
  - A Combiner class should be considered and implemented if the aggregation logic performed by the Reducer is associative and commutative, and if applying this aggregation locally within the Mapper's output phase can significantly reduce the volume of data shuffled across the network , and if you use it you should to show the output with running combiner and without combiner.
- **Custom Writable/WritableComparable (Conditional)**
  - Custom Writable or WritableComparable classes are required when the data types for keys or values are complex (e.g., multiple fields, custom objects) and cannot be adequately represented by standard Hadoop Writable types (like Text, IntWritable, LongWritable).

Section 1 — Distributed Cache Projects

In Distributed Cache projects, a small reference dataset is loaded into a HashMap during the mapper's setup() phase. Mappers then utilize this for O(1) in-mapper lookups to enrich or categorize each record.

Task 1: Employee Department Enrichment

You have a large employee activity log (employeeId, hoursWorked, date) and a small department lookup file (employeeId, departmentName). Load the lookup file into a HashMap during setup. For each activity record, look up the department and emit (departmentName, hoursWorked). The reducer calculates total hours worked per department.

Input & Output

File	Format	Example
Cache File	employeeId, departmentName	E001, Engineering
Input File	employeeId, hoursWorked, date	E001, 8, 2024-01-10
Output	departmentName totalHours	Engineering 24, Finance 8, HR 5

Requirements

- Load dept_lookup.txt into HashMap (key = employeeId, value = departmentName).
- Mapper: Emit (departmentName, hoursWorked); if employeeId is missing, emit 'UNKNOWN_'.
- Reducer: Implement CacheReducer to sum hoursWorked per departmentName.
- Driver: Implement CacheDriver to accept input/output paths and auto-delete existing output.
- Set reducers to 1; skip lines with non-integer hoursWorked (log a warning).

Task 2: Flight Route Enrichment

An airline has a flights log (flightId, routeCode, passengerCount, fare) and a small route reference file (routeCode, origin, destination, distanceKm). Load the reference into a HashMap during setup. For each flight, compute revenue = passengerCount * fare, emit (routeLabel, revenue+'|'+passengerCount). The reducer sums totalRevenue and totalPassengers per route.

Input & Output

File	Format	Example
Cache File	routeCode, origin, destination, distanceKm	RT01, Cairo, Dubai, 2415
Input File	flightId, routeCode, passengerCount, fare	FL001, RT01, 150, 320

File	Format	Example
Output	routeLabel totalRevenue totalPassengers	Cairo-Dubai 92800 290

Requirements

- Build routeMap: key = routeCode, value = origin-destination string.
- Mapper (CacheMapper): compute revenue; emit (routeLabel, revenue+'|'+passengerCount).
- Reducer (CacheReducer): parse composite value; accumulate totalRevenue and totalPassengers; emit tab-separated.
- Driver (CacheDriver): store cache path in Configuration; accept input/output paths.
- Handle unknown routeCode (emit 'UNKNOWN_'); skip lines where passengerCount or fare are not positive integers.

Task 3: Web Log URL Categorization

A web server produces high-volume access logs (requestId, urlPath, responseTimeMs, statusCode). A small URL-to-category file (urlPattern, category) classifies paths. Load it into a HashMap during setup. For each log line, emit (category, responseTimeMs+'|'+statusCode). The reducer computes: requestCount, avgResponseTimeMs (integer division), and errorCount (statusCode >= 400) per category.

Input & Output

File	Format	Example
Cache File	urlPattern, category	/api/products, API
Input File	requestId, urlPath, responseTimeMs, statusCode	R001, /api/products, 120, 200
Output	category requestCount avgResponseTimeMs errorCount	API 4 252 1

Requirements

- Load url_categories.txt into HashMap; unknown paths emit category 'OTHER'.
- Mapper (CacheMapper): emit (category, responseTimeMs+'|'+statusCode).
- Reducer (CacheReducer): accumulate requestCount, sumResponseTime, errorCount (statusCode >= 400); compute avgResponseTime; emit 3 tab-separated fields.
- Driver (CacheDriver): set reducers to 1; accept input/output paths.
- Skip lines where responseTimeMs or statusCode are not integers; log skipped count in cleanup().

Task 4: Store Region Revenue Lookup

A retail chain has daily sales records (saleId, storeId, amount) and a small store-to-region mapping (storeId, regionName). Load the mapping into a HashMap during setup. For each sale, look up the region and emit (regionName, amount). The reducer sums total revenue per region.

Input & Output

File	Format	Example
Cache File	storeId, regionName	ST01, North
Input File	saleId, storeId, amount	S001, ST01, 1500
Output	regionName totalRevenue	East 5900, North 6600, South 5800, West 2600

Requirements

- Load store_regions.txt into HashMap (key = storeId, value = regionName).
- Mapper (CacheMapper): emit (regionName, amount) as Text, IntWritable; unknown storeId emits 'UNKNOWN_'.
- Reducer (CacheReducer): sum amounts per regionName.
- Driver (CacheDriver): accept input/output paths; set reducers to 1; auto-delete existing output.
- Skip lines where amount is not a positive integer; log a warning per skipped line.

Task 5: Movie Genre Rating Analysis

A streaming platform stores user ratings (ratingId, movieId, userId, rating 1-5) and a small movie metadata file (movieId, title, genre, releaseYear). Load the metadata into a HashMap during setup. For each rating record, look up the genre and emit (genre, rating). The reducer calculates: ratingCount, avgRating (2 decimal places), and fiveStarCount per genre.

Input & Output

File	Format	Example
Cache File	movieId, title, genre, releaseYear	M001, The Grand Illusion, Drama, 1937
Input File	ratingId, movieId, userId, rating	RT01, M001, U10, 4
Output	genre ratingCount avgRating fiveStarCount	Comedy 3 4.00 1, Drama 5 4.00 2

Requirements

- Load movies.txt into HashMap (store genre at index 0).
- Mapper (CacheMapper): emit (genre, rating) as Text, IntWritable; emit 'UNKNOWN' for unrecognised movieIds.

- Reducer (CacheReducer): accumulate ratingCount, ratingSum, fiveStarCount; compute avgRating with String.format("%.2f",...); emit 3 tab-separated fields.
- Driver (CacheDriver): set reducers to 1.
- Reject ratings outside 1-5 with LOG.warn and a counter; log total processed/skipped in cleanup().

Section 2 — Reduce Side Join Projects

In Reduce Side Join projects, two input files are processed by different Mapper classes using MultipleInputs. Each mapper tags its output value with a prefix (e.g., customer~ or order~) so the reducer can distinguish the source.

Task 6: Order Customer Join

Join a customer file (customerId, customerName, city) with an orders file (orderId, customerId, productName, amount) to enrich each order with the customer's name and city. Emit one output line per order: customerName, city, productName, amount.

Input & Output

File	Format	Example
Customer File	customerId, customerName, city	C01, Alice, Cairo
Order File	orderId, customerId, productName, amount	O001, C01, Laptop, 15000
Output	customerId enriched order line	C01 Alice,Cairo,Laptop,15000

Requirements

- Configure job with cell.lkp.tag='customer~', trnx.tag='order~', and separator=',.
- CustomerMapper: key = customerId (Text); emit (customerId, 'customer~'+customerName+','+city).
- OrderMapper: emit (customerId, 'order~'+orderId+','+productName+','+amount).
- OrderReducer: collect customer info from 'customer~' tag; for each 'order~' record emit joined line; use 'UNKNOWN,UNKNOWN' for missing customers.
- Driver: use MultipleInputs; accept input/output paths; set reducers to 1.

Task 7: Employee Payroll Join

Join an employee directory (employeeId, firstName, lastName, department) with a monthly payroll file (payrollId, employeeId, month, baseSalary, bonus). For each payroll record, emit: firstName+' '+lastName, department, month, totalPay (baseSalary+bonus). The reducer also tracks maxPay across all months for each employee.

Input & Output

File	Format	Example
Lookup File	employeeId, firstName, lastName, department	EMP01, Nour, Hassan, Engineering
Transaction File	payrollId, employeeId, month, baseSalary, bonus	PR001, EMP01, Jan, 8000, 500
Output	employeeId fullName, dept, month, totalPay, maxPay	EMP01 Nour Hassan,Engineering,Jan,8500,8700

Requirements

- EmployeeMapper: emit (employeeId, 'emp~'+firstName+','+lastName+','+department).
- PayrollMapper: emit (employeeId, 'pay~'+month+','+baseSalary+','+bonus).
- PayrollReducer: collect employee record; compute totalPay = baseSalary+bonus per 'pay~' record; track maxPay; emit full enriched line.
- Driver: configure tags 'emp~' and 'pay~'; use MultipleInputs; set reducers to 1.
- Emit 'UNKNOWN EMPLOYEE,UNKNOWN' for unmatched employeeIds; skip payroll lines with non-integer salary/bonus.

Task 8: Course Enrollment Summary Join

Join a course catalog (courseId, courseName, instructor) with an enrollment file (enrollmentId, courseId, studentName, enrollmentDate) to produce an enriched enrollment report. For each enrollment record, emit: courseName, instructor, studentName.

Input & Output

File	Format	Example
Course Metadata	courseId, courseName, instructor	C001, Mathematics, Prof. Smith
Enrollment Records	enrollmentId, courseId, studentName, enrollmentDate	EN001, C001, Alice, 2024-01-15
Output	courseName, instructor, studentName	Mathematics, Prof. Smith, Alice

Requirements

- Use MultipleInputs.
- CourseMapper: emit (courseId, 'course~'+courseName+','+instructor).
- EnrollmentMapper: emit (courseId, 'enroll~'+studentName+','+enrollmentDate).
- Reducer: joins and emits the enriched enrollment details. Handle missing courses.

Section 3 — Partitioner Tasks

These projects focus on implementing custom Partitioner classes to distribute data across reducers based on specific criteria.

Task 9: Country-Based Partitioning

Analyze user activity data from multiple countries. Each reducer should process data from a single country only. Calculate the total number of active users and their activity count in each country.

Input & Output

Type	Details
Input	user_id, country, activity_count, registration_date U001,Egypt,150,2023-01-15 U002,Egypt,200,2023-02-20 U003,Saudi Arabia,180,2023-03-10 U004,UAE,220,2023-04-05 U005,Saudi Arabia,190,2023-05-12 U006,Kuwait,160,2023-06-18 U007,Egypt,210,2023-07-22 U008,Qatar,175,2023-08-30
Output	Egypt Users: 3 Total Activity: 560 Saudi Arabia Users: 2 Total Activity: 370 UAE Users: 1 Total Activity: 220 Kuwait Users: 1 Total Activity: 160 Qatar Users: 1 Total Activity: 175

Requirements

- Create a custom Partitioner that distributes data by country.
- Mapper: extract user_id and country.
- Reducer: calculate total activity per user per country.
- Set appropriate number of reducers (e.g., one per country).
- Handle large datasets (1GB+).

Task 10: Product Category Partitioning

Analyze sales data across multiple product categories. Each reducer should process one category only. Calculate total sales amount and number of transactions for each product in each category.

Input & Output

Type	Details
Input	product_id, category, sales_amount, transaction_count, date P001,Electronics,500,10,2023-01-15 P002,Electronics,750,15,2023-01-20 P003,Clothing,200,8,2023-02-10 P004,Clothing,300,12,2023-02-15 P005,Food,150,20,2023-03-05 P006,Books,180,5,2023-03-10 P007,Electronics,600,12,2023-03-15 P008,Sports,250,7,2023-03-20
Output	Electronics Total Sales: 1850 Transactions: 37 Clothing Total Sales: 500 Transactions: 20 Food Total Sales: 150 Transactions: 20 Books Total Sales: 180 Transactions: 5 Sports Total Sales: 250 Transactions: 7

Requirements

- Create a custom Partitioner that distributes data by product category.
- Mapper: extract product_id and category.
- Reducer: calculate total sales per product per category.
- Set appropriate number of reducers (e.g., one per category).
- Ensure even distribution of data across reducers.

Task 11: Department-Based Partitioning

Analyze employee data across multiple departments. Each reducer should process one department only. Calculate total salary, average salary, and number of employees in each department.

Input & Output

Type	Details
Input	employee_id, department, salary, position, hire_date E001,IT,5000,Developer,2020-01-15 E002,IT,6000,Senior Developer,2019-03-20 E003,HR,4000,HR Manager,2021-02-10 E004,Sales,4500,Sales Executive,2020-05-15 E005,Sales,5500,Sales Manager,2019-08-20 E006,Finance,5200,Accountant,2020-11-10 E007,IT,4800,Junior Developer,2022-01-05 E008,HR,3800,HR Assistant,2021-06-15
Output	IT Total: 15800 Avg: 5266 Employees: 3 HR Total: 7800 Avg: 3900 Employees: 2 Sales Total: 10000 Avg: 5000 Employees: 2 Finance Total: 5200 Avg: 5200 Employees: 1

Requirements

- Create a custom Partitioner that distributes data by department.
- Mapper: extract employee_id and department.
- Reducer: calculate salary statistics per department.
- Set appropriate number of reducers (e.g., one per department).
- Handle missing or invalid salary data.

Task 12: Region-Based Partitioning

Analyze store performance data across multiple geographic regions. Each reducer should process one region only. Calculate total visitors, average daily visitors, and number of stores in each region.

Input & Output

Type	Details
Input	store_id, region, visitors, daily_revenue, opening_date S001,North,500,2500,2020-01-15 S002,North,600,3000,2020-02-20 S003,South,400,2000,2020-03-10 S004,East,700,3500,2020-04-05 S005,West,550,2750,2020-05-12 S006,North,580,2900,2020-06-18 S007,South,450,2250,2020-07-22 S008,East,720,3600,2020-08-30
Output	North Total Visitors: 1680 Stores: 3 South Total Visitors: 850 Stores: 2 East Total Visitors: 1420 Stores: 2 West Total Visitors: 550 Stores: 1

Requirements

- Create a custom Partitioner that distributes data by region.
- Mapper: extract store_id and region.
- Reducer: calculate total visitors per store per region.
- Set appropriate number of reducers (e.g., one per region).
- Support large datasets (1GB+).

Task 13: Priority Level Partitioning

Analyze task management data with different priority levels. Each reducer should process one priority level only. Calculate the number of completed, pending, and cancelled tasks for each priority level.

Input & Output

Type	Details
Input	task_id, priority, status, assigned_to, due_date T001,High,Completed,John,2023-01-15 T002,High,Pending,Jane,2023-01-20 T003,Medium,Completed,Bob,2023-02-10 T004,Low,Completed,Alice,2023-02-15 T005,Medium,Pending,Charlie,2023-03-05 T006,High,Cancelled,David,2023-03-10 T007,Low,Pending,Eve,2023-03-15 T008,Medium,Completed,Frank,2023-03-20
Output	High Completed: 1 Pending: 1 Cancelled: 1 Medium Completed: 2 Pending: 1 Low Completed: 1 Pending: 1

Requirements

- Create a custom Partitioner that distributes data by priority level.
- Mapper: extract task_id and priority.
- Reducer: count tasks by status for each priority.
- Set appropriate number of reducers (e.g., one per priority level).
- Handle edge cases with unknown priority values.

Task 14: Semester-Based Partitioning

Analyze student academic performance data across different semesters. Each reducer should process one semester only. Calculate average grades, pass rates, and student statistics for each semester.

Input & Output

Type	Details
Input	student_id, semester, grade, subject, instructor ST001,Fall2023,85,Mathematics,Prof. Smith ST002,Fall2023,90,Mathematics,Prof. Smith ST003,Spring2024,88,Physics,Prof. Johnson ST004,Fall2023,92,Chemistry,Prof. Williams ST005,Spring2024,87,Biology,Prof. Brown ST006,Fall2023,78,English,Prof. Davis ST007,Spring2024,91,History,Prof. Miller ST008,Fall2023,85,Geography,Prof. Wilson
Output	Fall2023 Avg Grade: 86 Spring2024 Avg Grade: 88.7

Requirements

- Create a custom Partitioner that distributes data by semester.
- Mapper: extract student_id and semester.

- Reducer: calculate average grade per student per semester.
- Set appropriate number of reducers (e.g., one per semester).
- Handle grade validation (e.g., 0-100 range).

Task 15: Age Group Partitioning

Analyze demographic data across different age groups. Each reducer should process one age group only. Calculate income statistics, employment rates, and other demographic metrics for each age group.

Input & Output

Type	Details
Input	person_id, age, income, employment_status, education_level P001,25,3000,Employed,Bachelor P002,35,5000,Employed,Master P003,45,6000,Employed,Bachelor P004,22,2500,Student,High School P005,55,7000,Employed,PhD P006,28,4000,Employed,Bachelor P007,38,5500,Self-Employed,Master P008,62,6500,Retired,Bachelor
Output	20-30 Avg Income: 3166 31-50 Avg Income: 5500 51+ Avg Income: 6750

Requirements

- Create a custom Partitioner that distributes data by age group (e.g., 18-24, 25-34, 35-44, 45-54, 55+).
- Mapper: extract person_id and age.
- Reducer: calculate average income per age group.
- Set appropriate number of reducers (e.g., one per age group).
- Validate age values (must be positive and reasonable).

Task 16: Language-Based Partitioning

Analyze multilingual content data across different languages. Each reducer should process one language only. Calculate article count, author statistics, and content metrics for each language.

Input & Output

Type	Details
Input	article_id, language, author, publication_date, word_count A001,Arabic,Ahmed,2023-01-15,1500 A002,Arabic,Fatima,2023-02-20,1800 A003,English,John,2023-03-10,2000 A004,French,Marie,2023-04-05,1700 A005,Arabic,Ahmed,2023-05-12,1600 A006,Spanish,Carlos,2023-06-18,1900 A007,English,Jane,2023-07-22,2100 A008,German,Hans,2023-08-30,1800
Output	Arabic Articles: 3 English Articles: 2 French Articles: 1 Spanish Articles: 1 German Articles: 1

Requirements

- Create a custom Partitioner that distributes data by language.
- Mapper: extract article_id and language.
- Reducer: count articles per author per language.
- Set appropriate number of reducers (e.g., one per language).
- Handle unknown or unsupported languages.

Task 17: Status-Based Partitioning

Analyze order data with different statuses. Each reducer should process one status only. Calculate order count, total amount, and other metrics for each status (Pending, Completed, Cancelled, Processing).

Input & Output

Type	Details
Input	order_id, status, amount, customer_id, order_date O001,Pending,100,C001,2023-01-15 O002,Pending,150,C002,2023-01-20 O003,Completed,200,C003,2023-02-10 O004,Cancelled,50,C004,2023-02-15 O005,Completed,180,C005,2023-03-05 O006,Processing,120,C006,2023-03-10 O007,Pending,160,C007,2023-03-15 O008,Completed,190,C008,2023-03-20
Output	Pending Total: 410 Completed Total: 570 Cancelled Total: 50 Processing Total: 120

Requirements

- Create a custom Partitioner that distributes data by order status.
- Mapper: extract order_id and status.
- Reducer: calculate total amount per status.
- Set appropriate number of reducers (e.g., one per status).
- Validate amount values (must be positive).

Task 18: Time Slot Partitioning

Analyze employee attendance data across different time slots (Morning, Afternoon, Evening). Each reducer should process one time slot only. Calculate attendance rates and employee statistics for each time slot.

Input & Output

Type	Details
Input	employee_id, time_slot, present, department, date E001,Morning,Yes,IT,2023-01-15 E002,Morning,Yes,HR,2023-01-15 E003,Afternoon,Yes,Sales,2023-01-15 E004,Evening,No,IT,2023-01-15 E005,Morning,No,Finance,2023-01-15 E006,Afternoon,Yes,HR,2023-01-15 E007,Evening,Yes,Sales,2023-01-15 E008,Morning,Yes,IT,2023-01-15
Output	Morning Present: 3 Afternoon Present: 2 Evening Present: 1

Requirements

- Create a custom Partitioner that distributes data by time slot.
- Mapper: extract employee_id and time_slot.
- Reducer: count present employees per time slot.
- Set appropriate number of reducers (e.g., one per time slot).
- Handle invalid time slot values.

Task 19: Grade Level Partitioning

Analyze school data across different grade levels. Each reducer should process one grade level only. Calculate student count, teacher count, and educational metrics for each grade level.

Input & Output

Type	Details
Input	person_id, grade_level, role, name, joining_date P001,Grade1,Student,Alice,2020-01-15 P002,Grade1,Student,Bob,2020-02-20 P003,Grade2,Student,Charlie,2020-03-10 P004,Grade1,Teacher,Dr. Smith,2019-01-15 P005,Grade2,Teacher,Prof. Johnson,2019-02-20 P006,Grade3,Student,Diana,2020-04-05 P007,Grade1,Student,Eve,2020-05-12 P008,Grade2,Student,Frank,2020-06-18
Output	Grade1 Students: 3 Teachers: 1 Grade2 Students: 2 Teachers: 1 Grade3 Students: 1 Teachers: 0

Requirements

- Create a custom Partitioner that distributes data by grade level.
- Mapper: extract person_id and grade_level.
- Reducer: count students and teachers per grade.
- Set appropriate number of reducers (e.g., one per grade level).
- Parse grade level correctly from 'GradeX', etc.

Task 20: Server Location Partitioning

Analyze server performance data across different geographic locations. Each reducer should process one server location only. Calculate average response time, request count, and error rates for each location.

Input & Output

Type	Details
Input	request_id, server_location, response_time, status_code, timestamp R001,US-East,100,200,2023-01-15T10:30:00 R002,US-East,120,200,2023-01-15T10:31:00 R003,EU-West,150,200,2023-01-15T10:32:00 R004,Asia-Pacific,200,200,2023-01-15T10:33:00 R005,US-West,110,200,2023-01-15T10:34:00 R006,US-East,105,200,2023-01-15T10:35:00 R007,EU-West,155,500,2023-01-15T10:36:00 R008,Asia-Pacific,210,200,2023-01-15T10:37:00
Output	US-East Avg: 108.3 Requests: 3 EU-West Avg: 152.5 Requests: 2 Errors: 1 Asia-Pacific Avg: 205 Requests: 2 US-West Avg: 110 Requests: 1

Requirements

- Create a custom Partitioner that distributes data by server location.
- Mapper: extract request_id and server_location.
- Reducer: calculate average response time per location.
- Set appropriate number of reducers (e.g., one per location).
- Handle network errors and invalid response times.